

gnss-scope 実装設計 (202607)

実装の設計書 (構成・データモデル・実装上の注意)。役割分担は次のとおり。

文書	内容
funcspec-202607.md	機能仕様 (何をするか・仕様値)
usersGuide-202607.md	利用者の手引 (操作手順・画面の読み方)
design-202607.md (本書)	実装設計 (画面実装・データモデル・モジュール構成・リファクタ記録)
algospec-202607.md	算出式・自動判定ロジックの詳細
hardware-202607.md	機器・ファーム (Pico W / MAX-M10S / BLE) の仕様

本書は、前身 [gnss-tracker](#) (地理院地図の全画面表示がメイン) を [gnss-scope](#) として再構築した際の設計を出発点としている。受信パイプライン (BLE → 行復元 → NMEA パース → エポック確定 → 受信品質統計) の考え方は前身から引き継いでおり、**変更点は UI 構成・記録モデル・解析の対象**である。

1. 方針

- 地図をメインにしない。地図は5タブのうちの1つ。役割も「ライブ地図」から「load した記録の GNSS 値をマーカー表示する」へ移す (ライブ現在地表示は副機能として残す)。
- 記録は静止点のばらつき評価に一本化。1回の記録 = 1地点。旧「スナップショット記録 (1点だけ即保存)」は廃止し、[record](#) → [stop](#) → [save](#) に統一。旧「静的測位記録」の収集・集計・収束自動停止ロジックはそのまま流用する。
- 解析は [SkyPlot](#) / [SNR](#) / [DOP](#) / [DRMS](#) の4本立てとし、ライブと読込データの双方を同じ画面で見る。

2. 画面構成

ステータスバー (常設)	接続状態ドット / 状態 / REC / fix / 最終受信
ページ (タブで排他・全画面)	接続 / 記録 / 解析 / 地図 / 設定
タブバー	

旧構成 (地図全画面 + 下部ボタンで開くボトムシート) を廃止し、[.page](#) の排他表示に変更した ([js/tab-ui.js](#))。非表示中の [<canvas>](#) と [Leaflet](#) はサイズが 0 になるため、**タブ表示時に描き直す**：

タブ	表示時の処理
記録	recScatter.redraw()
解析	analysisUI.refresh() (現在のソースで全再描画)
地図	mapView.invalidateSize() + 読込データへフィット

タブ 表示時の処理

設定 `infoUI.refresh()` (アプリ更新の確認)

`CanvasView.redraw()` を追加し、「サイズ 0 のまま描いた分の描き直し」を共通化した。

3. 記録モデル (record → stop → save → load)

record —連続エポック収集→ stop —集計→ [未保存の記録 pending] —save→ IndexedDB

load ← |
▲
JSON ファイル —importer.js—

- **stop** は **DB に書かない**。集計結果を pending として返し、記録タブに散布図+集計テキストを表示する。停止理由 (`converged` / `timeout` / `maxEpochs` / `manual`) も pending に持つ。
- **save** で地点名 (既定 `yyyy-mm-dd-xx` の同日連番) とメモを付けて保存し、保存した記録をそのまま「読込データ」に設定する (解析タブ・地図タブへ即反映)。このとき調査日 (`surveyId`) と地点番号 (`pointNo`) を採番し、ツリーの根 `surveys` も用意する (4. 参照)。
- 収束自動停止 (`algospec-202607.md` 3.) は維持。自動停止も **stop** と同じ経路で pending になる。
- 記録中は Wake Lock を取得し、画面 OFF / バックグラウンドでは `visibilitychange` で収集を一時停止する。

サンプルに衛星リストを含める

保存サンプル (`toSample`) に `satellites: [{sys, prn, elev, azimuth, snr, used}]` を追加した。これにより **保存後・取込後でも任意エポックの SkyPlot / SNR を再現**できる (解析タブのスライダー)。容量は1エポックあたり衛星30個程度 = 数KB、120 エポックで数百KB程度に収まる。

4. データモデル (IndexedDB: `gnssScopeDB` v2)

DB 名は旧 `gnssDB` から変更した (旧アプリのデータと混ざらないようにするため)。

1日に何十地点もまわって2系統 (M10S の NMEA と Android の測位) を比べる使い方に合わせ、**調査日 → 地点 → 実データ**のツリーにしている。

```
surveys    '2026-07-31'                ← 根: 1日の調査
├─ sessions rec_<開始ms> + surveyId / pointNo  ← 枝: 1地点 (20地点なら20件)
│   └─ points  sessionId + surveyId / pointNo  ← 葉: その地点の実データ
│       └─ samples[]      M10S のパース済みエポック (30~120秒ぶん)
│       └─ rawNmea[]      M10S の生NMEA行
│       └─ deviceSamples[] Android (端末内蔵GNSS) の測位
```

ストア	キー	内容
surveys	id (yyyy-mm-dd)	{ id, label, memo, createdAt, updatedAt }
sessions	id (bySurvey index)	{ id, type:'record', surveyId, pointNo, label, memo, createdAt, endedAt, window, summary, (importedAt, sourceId, sourcePointNo) }
points	id (bySession index)	{ id, sessionId, surveyId, pointNo, kind:'record', stats, samples[], (rawNmea[], deviceSamples[], deviceStats) }
settings	key	uere / autoStop / minSec / maxSec / maxEpochs / deviceGnss / saveRawNmea / mapType / trackEnabled / tileCacheMeta - summary: { lat, lon, altMSL, count, drms, cep50, cep95, stopReason, rxStats, rawLines, rawTruncated, (deviceDrms, deviceCount) } - stats: computeStaticStats() の戻り値 (中心・中央値・東西/南北標準偏差・DRMS/2DRMS・CEP50/CEP95・標高平均/標準偏差・fix内訳・平均DOP・平均衛星数・平均C/N0・散布図オフセット) - 設定キーは旧 staticMinSec などから minSec 等へ改名 (記録が静的測位専用ではなくなったため)。

地点の対応をどう保証するか

2系統を同じ point レコードに入れるのが第一の担保で、これで「どの NMEA とどの Android が対か」は構造上ずれない。その上で、後から見て「本当に同じ時間に測ったか」を確認できるよう session.window (js/survey.js の buildWindow) を残す。

フィールド	内容
startedAt / endedAt / durationSec	record → stop の記録区間 (端末時計)
gnss	NMEA エポックが実際に取れていた範囲 {startedAt, endedAt, durationSec, count}
gnssUtc	同じ範囲を GPS時刻(UTC)で見たもの
device	Android の測位が取れていた範囲
overlap	{ overlapSec, coverGnss, coverDevice, deviceInRecording, deviceTotal }
clockOffsetMs	端末時計 - GPS時刻。2系統を同じ時間軸へ並べ直すときの補正量
deviceLagMs	Android の 受信時刻 - 測位確定時刻。古い fix を返していないかの目安

時刻軸の扱いに注意が要る。M10S のサンプルは t=GPS時刻 / recvAt=端末時計、Android は t=OS が測位を確定した時刻 / recvAt=アプリが受け取った時刻で、**t どうしは別の時計** (Android は maximumAge:0 でも数秒前の fix を返すことがある)。区間の突き合わせは両者が共通に持つ recvAt で行い、t とのズレは clockOffsetMs / deviceLagMs として別に残す。

`overlap.deviceInRecording` は記録区間を基準に数える。NMEA の最初のエポックは複数センテンスが揃うまで確定せず1秒強かかるため、`gnss` の範囲で数えると記録開始直後に届いた Android のサンプルを取りこぼす。また、静止中は OS が更新を間引いて Android が1〜数点しか来ないことがあり、割合 (`coverDevice`) だけでは対応を判断できない。

地点番号 `pointNo` は同じ `surveyId` の中の連番で、欠番は再利用しない (最大値+1)。一度出した対応表と番号がずれないようにするため。

生NMEA行の保存

設定 `saveRawNmea` (既定 ON) のとき、受信した行を `point.rawNmea = [{ t, line }]` にそのまま残す。パースを通す前に分岐させるので、チェックサムNG行や `$PPICO` も含まれる (「受信したそのまま」を残す意味)。後から RTKLIB などの別ツールで再解析できる。

容量は 1Hz・10〜20行/秒で1地点 (30〜120秒) あたり 100〜150KB、20地点/日で 2〜3MB 程度。上限 20000 行 (`MAX_RAW_LINES`) を超えたぶんは捨て、本数を `summary.rawTruncated` に残して記録が途中までであることを隠さない。

v1 → v2 の移行

`onupgradeneeded` の `versionchange` トランザクション内で完結させる (`migrateToV2`)。 `surveys` ストアと `sessions.bySurvey` を作り、既存の記録には `createdAt` のローカル日付から `surveyId` を、同じ日の中では `createdAt` 昇順で `pointNo` を採番して `points` にも複写する。これで過去データもそのまま調査日ツリー・対応表に載る。

端末内蔵GNSS の並行取得 (比較データ)

`deviceSamples` / `deviceStats` / `summary.deviceDrms` は 並行取得した記録にだけ足す (取得していない記録の形は従来のまま=後方互換。取込側も任意扱いで読む)。

- `deviceStats` は `computeDeviceStats()` の戻り値 = `computeStaticStats()` の全項目 + `avgAccuracy` / `dupCount` / `offsetFromRef {e, n, distM, bearingDeg}`。ばらつきの定義を M10S と共有するため、集計本体は `computeStaticStats` をそのまま使う。
- `offsetFromRef` の基準は **M10S の中心**。散布図で共通原点に重なる際の平行移動量そのものなので、描画側は差分を再計算せず `offsetFromRef.e / .n` を足すだけでよい。
- `dupCount` (直前と完全同一座標の点数) は保存後・取込後も再計算できるよう サンプル列から算出する (フラグとしてサンプルに持たせない)。

エクスポート / インポート

地点ごと (`file-io.js`) :

- CSV: 1行1エポック。列に `snr_avg_used_dbhz` (使用衛星の平均 C/N0) と `recv_at_local` を持つ。末尾に地点の識別 (`survey_id / point_no`) と集計値をコメント行で付ける。内蔵GNSS は行として混ぜず、`# device_*` のコメント行で比較値のみ出す。
- GPX: 記録の中心を `wpt`、生エポック群を `trk` として出力 (M10S のみ)。
- NMEA: `point.rawNmea` の行をそのまま `.nmea` として出力 (先頭に地点を示すコメント行)。
- JSON: `{ app:'gnss-scope', format:1, exportedAt, session, point }`。

調査日ごと (1日20地点をまとめて持ち出す) :

- 対応表CSV: **1行1地点**で M10S と Android を横並びにした表 (`compareRow / COMPARE_HEADER`)。先頭2列が `survey_id / point_no` で、これが地点の対応そのもの。ばらつき (DRMS / CEP / 標準偏差) に加え、`center_offset_m · drms_ratio_dev_over_gnss`、対応の検証列 (`overlap_s / cover_*_pct / dev_in_record_window / clock_offset_ms / dev_fix_lag_ms`) を持つ。2系統比較の入口はこのファイル。
- バンドルJSON: `{ app:'gnss-scope', format:2, exportedAt, survey, points:[{session, point}] }`。`point` を丸ごと入れるので生NMEA・生エポック・Android の測位が地点ごとに揃ったまま出る。

取込 (`importSessionFile`) は単体 (format 1)・バンドル (format 2) を同じ入口で受け、常に `{session, point}` の配列を返す。取込時は **必ず新 id を採番**し (同じファイルの二重取込でも上書きしない)、元 id は `sourceId`、元の地点番号は `sourcePointNo` に残す。調査日は元データの日付をそのまま使い (同じ日の測定は同じ調査日に集まる方が突き合わせやすい)、地点番号だけ取込先で採り直す。`stats` が欠けている JSON は `computeStaticStats` で再計算する。

5. 解析タブ

表示	ライブ	読込データ
SkyPlot / SNR	最新エポック	スライダで選んだエポック (保存済み衛星リストから再現)
DOP (バー+数値・測位モード・衛星数・精度ソース・GST σ)	最新エポック	同上
DRMS (散布図+集計テキスト)	記録中／直近の記録の 暫定集計	読込データの確定集計
DRMS の比較系列 (端末内蔵GNSS)	同区間の暫定集計 (記録中から見える)	<code>point.deviceStats</code>

- レイアウトは2列グリッド (`.an-grid`)。上段 SkyPlot / DRMS、下段 SNR / DOP。見出し (`.sub-head`) と本体 (`.an-cell`) を別セルにして、見出しの行数が違ってても左右の図の上端が揃うようにしている。行数の多い DRMS 集計テキストは `.an-wide` で2列ぶち抜き。列幅に追従させるため `grid-template-columns` は `minmax(0, 1fr)` (canvas の内在サイズに列幅を引っ張られないように)。
- ソースはラジオで切替。読込データが無い間は「読込データ」を `disabled` にする。
- 保存サンプル → 表示用エポックへの変換 (`sampleToEpoch`) で `recvAt` を補い、`canvas-view.js` のキャリーフォワード判定 (`holdDecision`) がライブと同じ経路で働くようにしている。
- 系統別内訳は、ライブでは GSA 由来の `usedBySys / inViewBySys` を使い、無い場合 (読込データ) は衛星リストから数える。
- 比較系列は `ScatterPlotView.update(stats, deviceStats)` の第2引数で渡す。基底クラスのリサイズ再描画は `update(this._last)` と1引数で呼ぶため、第2引数の既定値を **保持中の系列** (`this._lastDevice`) にしてある (比較を消すときは明示的に `null` を渡す)。
- 統計円のラベル (CEP50 / DRMS / 内蔵) は象限を分けて描く。比較系列を重ねるとスケールが大きい方に引かれ、M10S 側の円が小さくなってラベルが中心で潰れるため。中心からの距離にも下限 (10 px) を設けている。

6. モジュール構成

「1機能＝1ファイル」を基準に、受信 / 解析・記録 / 画面の3層で分ける（202607 のリファクタリングで 27 ファイル → 17 ファイルに統合。詳細は 9.）。

層	モジュール	責務
エントリ	<code>app.js</code>	受信パイプラインと各タブの結線のみ
共通	<code>view-utils.js</code>	\$ (DOM ショートハンド)・タブ切替・表示フォーマッタ
受信	<code>transport.js</code>	BLE(NUS) 接続・自動再接続 + 開発用モック（同じ <code>onFrame I/F</code> ）
受信	<code>nmea.js</code>	行復元（ <code>LineBuffer</code> ）+ センテンス解析 + チェックサム（ <code>xorChecksum</code> ）
受信	<code>epoch.js</code>	同一時刻センテンス群 → 1エポック
受信	<code>stream-stats.js</code>	受信品質統計（\$PPICO 突合を含む）
受信	<code>device-gnss.js</code>	端末内蔵GNSS（Geolocation API）の並行取得。記録中のみ <code>watchPosition</code>
解析・記録	<code>accuracy.js</code>	水平精度推定・DRMS/CEP 集計・収束判定
解析・記録	<code>charts.js</code>	Canvas 描画（共通土台 + スカイプロット / SNR / 散布図）
解析・記録	<code>recorder.js</code>	record / stop / save の制御
解析・記録	<code>survey.js</code>	調査日→地点のID体系・測定区間・対応検証（純粋関数のみ）
解析・記録	<code>storage.js</code>	IndexedDB ラッパ（v1→v2 移行を含む）
解析・記録	<code>file-io.js</code>	CSV / GPX / NMEA / JSON 出力・調査日バンドルと対応表・JSON 取込
画面	<code>connect-ui.js</code>	接続タブ（受信経路のライフサイクル・受信品質パネル・モック切替）
画面	<code>record-ui.js</code>	記録タブ（record/stop/save/load・一覧・Wake Lock）
画面	<code>analysis-ui.js</code>	解析タブ（SkyPlot / SNR / DOP / DRMS、ライブ↔読込切替）
画面	<code>map.js</code>	地図タブ（ <code>MapView</code> + タブ配線）
画面	<code>settings-ui.js</code>	設定タブ（各種設定 + アプリ更新確認）
画面	<code>tile-cache.js</code>	オフラインタイル事前DL（ <code>TileCache</code> + 設定タブ内のUI）

- `app.js` は結線のみ。DOM 操作はタブごとの `*-ui.js` が持つ。
- 読込データ（loaded）は `app.js` が保持し、`setLoaded()` で解析タブ・地図タブへ配る。
- `device-gnss.js` は「受信経路」であってライフサイクルを持たないため、開始/停止は `Recorder` が握る（記録区間＝取得区間を自明にする）。`app.js` は `onSample` → `recorder.addDeviceSample` /

`onStatus` → `recordUI.onDeviceStatus` を結線するだけ。

- 廃止：旧「ライブ」タブ（記録タブの「現在の測位値」へ統合）／旧「情報」タブ（設定タブへ統合）／スナップショット記録。

7. PWA・オフラインタイトル

- 版数は `sw.js` の `APP_VERSION` (`'yyyy-mm-dd.n'` = 更新日 + 同日内の連番。日付が変われば 1 に戻る)。シェルキャッシュ名は `gnss-scope-shell-${APP_VERSION}`（接頭辞は `activate` の掃除で他キャッシュと区別するためのもの）。`settings-ui.js` は `APP_VERSION` を正規表現で読み取り、接頭辞なしの版数で表示・比較する（旧SWがキャッシュ名を返す場合は接頭辞を外す）。`APP_SHELL` には全モジュールを列挙する。
- 地理院タイトルは `cache-first` でキャッシュ `gsi-tiles` に格納する。Service Worker（実行時の取得）と事前ダウンロード（`tile-cache.js`）が同じキャッシュ名を共有する点が必要。
- 未キャッシュのタイトルをオフラインで要求した場合は 504 を返し、Leaflet 側でグレー表示にする。

タイトルマニフェスト（`data/tile_manifest.json`）

事前ダウンロードの対象を座標リストで持つ（Format A）。マニフェストは `data/` を唯一の置き場とし、ルート直下に複製を置かない。

```
{
  "version": "2026-06-01",
  "layers": {
    "<任意のキー>": { "z": 17, "tiles": [[x, y], [x, y], ...] }
  }
}
```

- `z` はズームレベル、`tiles` は当該ズームのタイトル座標 `[x, y]` の配列。
- URL は選択中の地図種別のテンプレート（`GSI_LAYERS[mapType].url`）の `{z}/{x}/{y}` を置換して生成する。したがって地図種別を変えると、その種別のタイトルは再取得が必要。
- `version` は取得時に `settings.tileCacheMeta` へ記録し、オフラインでも取得済み内容を表示できるようにする。

8. リファクタリング（202607・機能変更なし）

ファイル数が多く（27）、似た処理が分散していたため、機能を変えずに統合・整理した。

8-1. ファイルの統合（27 → 17）

統合後	統合したファイル	まとめた理由
<code>nmea.js</code>	<code>nmea-parser</code> + <code>line-buffer</code>	どちらも「生バイト列 → 1センテンス」の下位層。チェックサム計算を共用できる
<code>transport.js</code>	<code>ble-client</code> + <code>mock-feeder</code>	同じ <code>onFrame</code> I/F で差し替える受信経路。実機／モックは対の関係
<code>epoch.js</code>	<code>epoch-assembler</code> （改名）	名前をファイルの役割に合わせた

統合後	統合したファイル	まとめた理由
<code>charts.js</code>	<code>canvas-view</code> + <code>sky-plot</code> + <code>snr-chart</code> + <code>scatter-plot</code>	すべて Canvas 描画。共通土台と3ビューを同居させ、共通ヘルパを内部化
<code>file-io.js</code>	<code>exporter</code> + <code>importer</code>	記録のファイル入出力。JSON 形式の定義を1か所に持てる
<code>view-utils.js</code>	<code>format</code> + <code>tab-ui</code> + \$ ヘルパ	画面共通のユーティリティ。\$ を各モジュールへ引数で配る配線が不要になる
<code>map.js</code>	<code>map</code> + <code>map-ui</code>	地図本体とタブ配線は常に一緒に変更する
<code>tile-cache.js</code>	<code>tile-cache</code> + <code>tile-ui</code>	タイル取得と、その進捗UIは一体
<code>settings-ui.js</code>	<code>settings-ui</code> + <code>info-ui</code>	アプリ更新確認は設定タブの中にある機能
<code>record-ui.js</code>	<code>record-ui</code> + <code>wake-lock</code>	「記録中の画面維持」は記録機能の一部
<code>connect-ui.js</code>	(<code>app.js</code> から分離・新規)	接続制御と受信品質パネルを <code>app.js</code> から切り出し

`app.js` は結線の上に絞り 330 行 → 162 行。各 UI モジュールは自分が使う描画ビューを 自前で生成するようにして、`app.js` からの受け渡しを減らした。

8-2. 重複処理の共通関数化

対象	変更
チェックサム XOR 計算	パーサの検証とモックの文生成で重複 → <code>xorChecksum()</code> に集約
HTML / XML エスケープ	<code>escapeHtml</code> と <code>escXml</code> の2実装 → <code>escapeMarkup()</code> に統合
キャリーフォワード判定	スカイプロットと SNR で同じ 12 行 → <code>CanvasView._holdOrClear()</code>
十字・リング・方位ラベル描画	スカイプロットと散布図で別実装 → <code>drawCross</code> / <code>strokeCircle</code> / <code>fillCircle</code> / <code>drawCompass</code>
タイル URL 列挙	件数カウントとダウンロードで二重ループ → <code>TileCache.tileUrls()</code>
設定行の保存処理	6 個のほぼ同型リスナ → 定義テーブル + 2 ループ
「使用/視野内」表示	記録タブと解析タブで重複 → <code>satsText()</code>
追従トグルの通知	3 か所で個別に UI 反映 → <code>MapView.setFollow()</code> に集約
一覧の行アクション	if 連結 → <code>runAction()</code> + エクスポートの対応表

8-3. 未使用コードの削除

- エポックが持っていて誰も読まない値：`geoidSep` / `status` (RMC) / `altStd` (GST) / `sentenceTypes` / `invalidCount` (不正文の件数は `stream-stats.js` が数えている)。パーサ側の対応フィールドも併せて削除。

- 使われていない設定可能性：NmeaBle の `deviceName` / `maxBackoffMs`（モジュール定数へ）、MockFeeder の `hz`、TileCache の `zoomFilter` と `countTiles()`。
- 読めないフィールド・引数：NmeaBle.`status` と `_setStatus()`、WakeLockManager の `isSupported()` と `onChange` の第1引数、SNR チャートの到達不能ガード。
- 外へ出す必要のない export の内部化：HOLD_MS / CanvasView / `stopReasonText` / STOP_REASON / `rxStatsText` / JSON_FORMAT。

なお `computeStaticStats` の `avgPdop` / `avgVdop` は画面に出ないが、**JSON エクスポートで利用者へ渡る値**なので残している。

9. テスト

`node tools/check-syntax.mjs` に以下を追加した。

- 使用衛星の平均 C/N0（未使用衛星を除外すること）
- `formatStats` が未保存の記録でも生成できること
- 記録フロー：`stop` で集計しても DB に書かないこと／サンプルに衛星リストが残ること／`save` で `sessions`・`points` に保存され `summary` に集計値が入ること
- 端末内蔵GNSS の比較：`offsetBetween` の距離と方位／`computeDeviceStats` の座標重複数・平均 accuracy・中心ズレ／`formatCompare` の主要行と過小評価の注意／`withDevice` の ON・OFF で取得の開始停止と保存形（OFF なら従来どおりのキー構成）が変わること